

Design of a Car Control System

1) Task set (single core, preemptive)

Task (τ_i)	Function	WCET C_i	Period T_i	Deadline D_i	Notes
τ_1	Speed measurement	4 ms	20 ms (50 Hz)	5 ms	Constrained deadline ($D < T$)
τ_2	ABS control	10 ms	40 ms (25 Hz)	40 ms	Control law update
τ_3	Fuel injection	40 ms	80 ms (12.5 Hz)	80 ms	Actuation scheduling
—	Other SW (audio, A/C...)	best-effort	—	soft	Runs on leftover CPU

Assumptions (design conditions): single CPU, high-res timers, fully preemptive; shared data protected with SRP/PCP¹; ISR work kept minimal (timestamp, buffer) with bottom-half jobs doing the heavy lifting; context-switch + scheduler overhead $\ll 1$ ms.

2) Pick the scheduling policy

Because τ_1 has $D_1 < T_1$, use **Deadline-Monotonic (DM)** (fixed-priority by shorter relative deadlines):

Priority order $\rightarrow \tau_1$ (5 ms) $>$ τ_2 (40 ms) $>$ τ_3 (80 ms) $>$ soft tasks (background/server).

3) Quick feasibility checks

- Total hard-real-time utilization:

$$U = \sum C_i / T_i = 4/20 + 10/40 + 40/80 = 0.20 + 0.25 + 0.50 = 0.95.$$

- RM utilization bound for $n=3$ is $(2^{1/3} - 1) \approx 0.779 \rightarrow$ **fails** the simple RM bound (not decisive).
- **Response-Time Analysis (RTA) under DM** (fully preemptive, no blocking):

¹ In computer science and real-time systems, **SRP** (Stack Resource Policy) and **PCP** (Priority Ceiling Protocol) are resource-sharing and synchronization protocols used in multi-tasking operating systems to prevent unbounded priority inversion and avoid deadlocks. They manage access to shared data and system resources safely.

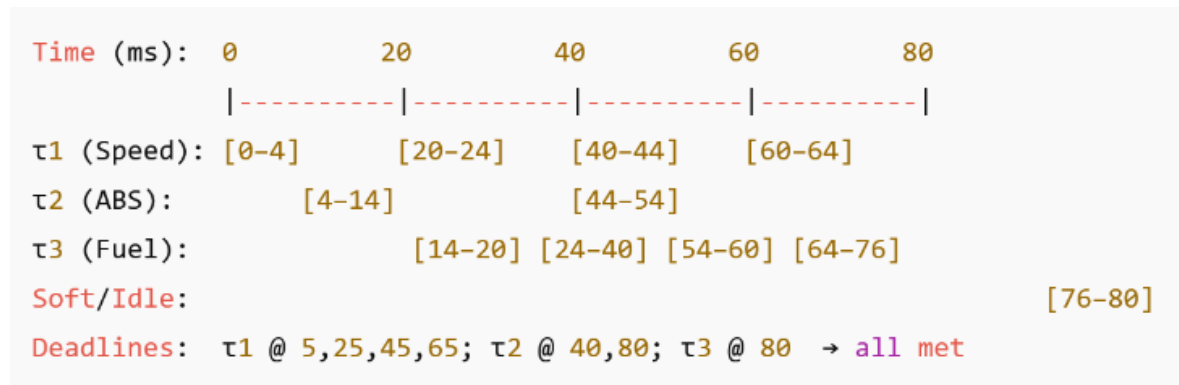
- $\tau_1: R_1 = C_1 = 4 \text{ ms} \leq D_1 = 5 \text{ ms} \checkmark$
- $\tau_2: R_2 = 10 + \lceil R_2/20 \rceil \cdot 4 \rightarrow \text{iterate: } 10 \rightarrow 14 \rightarrow \mathbf{14 \text{ ms}} \leq 40 \text{ ms} \checkmark$
- $\tau_3: R_3 = 40 + \lceil R_3/20 \rceil \cdot 4 + \lceil R_3/40 \rceil \cdot 10$
 $40 \rightarrow 58 \rightarrow 72 \rightarrow \mathbf{76 \text{ ms}} \leq 80 \text{ ms} \checkmark$

All hard tasks meet deadlines under DM despite $U = 0.95$.

Slack (per job): τ_1 : **1 ms**, τ_2 : **26 ms**, τ_3 : **4 ms**. CPU slack over hyperperiod is $\approx$ **5%**.

4) One hyperperiod schedule (80 ms)

Hyperperiod $LCM(20,40,80) = 80ms$. Worst-case phasing at $t = 0$:



This makes it clear soft tasks have $\sim 4\text{ ms}$ every 80 ms ($\approx 5\%$) at minimum; they can also run opportunistically whenever τ_3 is not pending.

5) Integrating the “soft” tasks safely

Soft workloads (audio, A/C UI, logging) should **never** jeopardize hard deadlines. Two safe patterns:

- **Background priority** (lowest fixed priority): they run only when no hard task is ready.
- **Server-based reservation** (recommended): e.g., Sporadic/Deferrable Server with budget **Q=4 ms**, period **T=80 ms** (5%). This caps interference deterministically and smooths bursts.

If the infotainment load needs >5% CPU, consider: (a) tightening WCETs (esp. τ_3), (b) relaxing non-safety periods, or (c) a second core/SoC partition for non-critical apps.

Design Summary: Basic design procedure

1. Requirements & timing model

Define for each function: T_i (sampling), D_i (deadline), C_i (WCET), release jitter, precedence, I/O latencies.

2. Measure/estimate WCET

Measurement-based with stress inputs + cache/branch pessimism, or static analysis; include ISR and driver costs.

3. Choose policy & assign priorities

Constrained deadlines → **DM**; otherwise RM or EDF. Reserve budget for soft tasks via a server if needed.

4. Account for resource sharing

Use **SRP/PCP** to prevent priority inversion. Compute blocking B_i and re-run RTA with $R_i = C_i + B_i + \sum \lceil R_i/T_j \rceil C_j$.

5. Schedulability analysis

- Fixed-priority: exact **RTA** as above.
- EDF (if chosen): demand-bound test (DBF) for constrained deadlines. Include scheduler/context-switch overheads and worst-case phasing.

6. Design the time base

Prefer tickless/high-res timers. With $D_1 = 5ms$, keep scheduling granularity $\ll 1$ ms and ISR execution short ($\lesssim 50 - 100 \mu s$).

7. Map to architecture

Split ISR (top-half) vs task (bottom-half). Ensure end-to-end sensor→compute→actuator delay fits control needs (e.g., $\leq D$ for τ_2/τ_3), and bound jitter for τ_1 sampling.

8. Implementation details

Pin critical code/data, avoid dynamic allocation in hard tasks, bound critical sections ($< a few 100 \mu s$), size stacks, and log WCET margins.

9. Verification on target

Instrument timestamps, measure worst-case response & release jitter, run overload and fault-injection tests; confirm the hyperperiod pattern and slack windows.

10. Refine & guard

Add overload handlers (shed soft work), health monitoring, and watchdogs; revisit $C_i/T_i/D_i$ if margins are thin (notably τ_1 and τ_3 here).

7) Practical tuning tips for this set

- Biggest risk is **tight margins on τ_1 (1 ms) and τ_3 (4 ms)**. If you can shave τ_3 from 40 ms $\rightarrow$ 36 ms, R_3 drops to ~ 72 ms and system slack doubles to $\sim 10\%$.
- Consider small **release offsets** to reduce contention (e.g., start ABS a few ms after τ_1 completes) while keeping periodicity and DM priorities.